Version: 1.0



Selection

C-Algorithmiques-Fondamentaux

Résumé

Ce projet introduit les concepts algorithmiques fondamentaux en C, en mettant l'accent sur la récursivité, les calculs mathématiques et les techniques de résolution de problèmes.

#C #Algorithmes #Récursivité



Avertissement relatif à la propriété intellectuelle

L'ensemble du contenu présenté dans ce module de formation — y compris, sans s'y limiter, les textes, images, graphiques et autres éléments — est protégé par les droits de propriété intellectuelle détenus par l'Association 42.

Conditions d'utilisation

- **Usage personnel :** L'utilisation du contenu de ce module est strictement réservée à un usage personnel. Toute reproduction, diffusion, modification, utilisation publique ou commerciale est interdite sans l'autorisation écrite préalable de l'Association 42.
- **Respect de l'intégrité :** Il est interdit de modifier, altérer ou adapter le contenu de manière à en dénaturer le sens ou à porter atteinte à son intégrité.

Protection des droits

Toute violation des présentes conditions constitue une atteinte aux droits de propriété intellectuelle et peut faire l'objet de poursuites. L'Association 42 se réserve le droit d'engager toute action nécessaire à la protection de ses droits, notamment par voie judiciaire et en réclamant des dommages et intérêts le cas échéant.

Pour toute question relative à l'utilisation du contenu ou pour obtenir une autorisation, veuillez contacter : `legal@42.fr`

Table des matières

1 Instructions 1

2 Préambule 5

3 Tutoriel 6

4 Exercice 0 : ft_iterative_factorial 9

5 Exercice 1 : ft_recursive_factorial 10

6 Exercice 2 : ft_iterative_power 11

7 Exercice 3 : ft_recursive_power 12

8 Exercice 4 : ft_fibonacci 13

9 Rendu et évaluation par les pairs 14

Chapitre 1

Instructions

- Seule cette page servira de référence : ne faites pas confiance aux rumeurs.
- Attention ! Ce document pourrait potentiellement changer jusqu'à la soumission.
- Assurez-vous d'avoir les permissions appropriées sur vos fichiers et répertoires.
- Vous devez suivre les procédures de soumission pour tous vos exercices.
- Vos exercices seront vérifiés et notés par vos camarades de classe.
- De plus, vos exercices seront vérifiés et notés par un programme appelé Moulinette.
- Moulinette est très méticuleuse et stricte dans son évaluation de votre travail. Elle est entièrement automatisée, et il n'y a aucun moyen de négocier avec elle. Donc, pour éviter les mauvaises surprises, soyez aussi minutieux que possible.
- Moulinette n'est pas très ouverte d'esprit. Elle n'essayera pas de comprendre votre code s'il ne respecte pas la Norme. Moulinette s'appuie sur un programme appelé `norminette` pour vérifier si vos fichiers respectent la norme. En bref : il serait stupide de soumettre un travail qui ne passe pas la vérification de `norminette`.
- Ces exercices sont soigneusement disposés par ordre de difficulté - du plus facile au plus difficile. Nous `ne considérerons pas` un exercice plus difficile réussi si un exercice plus facile n'est pas parfaitement fonctionnel.
- Utiliser une fonction interdite est considéré comme de la tricherie. Les tricheurs obtiennent -42, et cette note est non négociable.
- Vous n'aurez à soumettre une fonction main() que si nous demandons un programme.
- Moulinette compile avec ces flags : -Wall -Wextra -Werror, et utilise `cc`.
- Si votre programme ne compile pas, vous obtiendrez `0`.
- Vous ne pouvez pas laisser aucun fichier supplémentaire dans votre répertoire autre que ceux spécifiés dans le sujet.
- Vous avez une question ? Demandez à votre pair à droite. Sinon, essayez votre pair à gauche.
- Votre guide de référence s'appelle `Google / man / Internet / ...`.
- Consultez le Slack Piscine.
- Examinez attentivement les exemples. Ils pourraient très bien appeler à des détails qui ne sont pas explicitement mentionnés dans le sujet...

© 2026 Association 42. - Tous droits réservés

- Par Odin, par Thor ! Utilisez votre cerveau ! ! !



N'oubliez pas d'ajouter l'*en-tête standard 42* dans chacun de vos fichiers .c/.h. Norminette vérifie son existence de toute façon !



Norminette doit être lancée avec le flag *-R CheckForbiddenSourceHeader*. Moulinette l'utilisera aussi.

© 2026 Association 42. - Tous droits réservés

Contexte

La Piscine C est intense. C'est votre premier grand défi à 42 — une plongée profonde dans la résolution de problèmes, l'autonomie et la communauté.

Durant cette phase, votre objectif principal est de construire vos fondations — à travers la lutte, la répétition, et surtout l'échange d'**apprentissage par les pairs**.

À l'ère de l'IA, les raccourcis sont faciles à trouver. Cependant, il est important de considérer si votre utilisation de l'IA vous aide vraiment à grandir — ou si elle se contente de vous empêcher de développer de vraies compétences.

La Piscine est aussi une expérience humaine — et pour l'instant, rien ne peut remplacer cela. Pas même l'IA.

Pour un aperçu plus complet de notre position sur l'IA — en tant qu'outil d'apprentissage, dans le cadre du programme ICT, et en tant qu'attente croissante sur le marché du travail — veuillez vous référer à la FAQ dédiée disponible sur l'intranet.

Message principal

- Construire des fondations solides sans raccourcis.
- Développer réellement des compétences techniques et personnelles.
- Expérimenter un véritable apprentissage par les pairs, commencer à apprendre à apprendre et à résoudre de nouveaux problèmes.
- Le parcours d'apprentissage est plus important que le résultat.
- Apprendre les risques associés à l'IA, et développer des pratiques de contrôle efficaces et des contre-mesures pour éviter les pièges courants.

Règles pour les apprenants :

- Vous devez appliquer le raisonnement à vos tâches assignées, surtout avant de vous tourner vers l'IA.
- Vous ne devez pas demander de réponses directes à l'IA.
- Vous devez apprendre l'approche globale de 42 sur l'IA.

© 2026 Association 42. - Tous droits réservés

Résultats de la phase :

Dans cette phase de fondation, vous obtiendrez les résultats suivants :

- Acquérir des bases techniques et de codage appropriées.
- Savoir pourquoi et comment l'IA peut être dangereuse pendant cette phase.

Commentaires et exemple :

- Oui, nous savons que l'IA existe — et oui, elle peut résoudre vos activités. Mais vous êtes ici pour apprendre, pas pour prouver que l'IA a appris. Ne gaspillez pas votre temps (ou le nôtre) juste pour démontrer que l'IA peut résoudre le problème donné.
- Apprendre à 42 ne consiste pas à connaître la réponse — il s'agit de développer la capacité à en trouver une. L'IA vous donne la réponse directement, mais cela vous empêche de construire votre propre raisonnement. Et le raisonnement prend du temps, des efforts, et implique des échecs. Le chemin vers le succès n'est pas censé être facile.
- Gardez à l'esprit que lors des examens, l'IA n'est pas disponible — pas d'internet, pas de smartphones, etc. Vous réaliserez rapidement si vous avez trop compté sur l'IA dans votre processus d'apprentissage.
- L'apprentissage par les pairs vous expose à différentes idées et approches, améliorant vos compétences interpersonnelles et votre capacité à penser de manière divergente. C'est bien plus précieux que de simplement discuter avec un bot. Alors ne soyez pas timide — parlez, posez des questions, et apprenez ensemble !
- Oui, l'IA fera partie du programme — à la fois comme outil d'apprentissage et comme sujet en soi. Vous aurez même l'occasion de construire votre propre logiciel d'IA. Afin d'en apprendre davantage sur notre approche crescendo, vous passerez par la documentation disponible sur l'intranet.

✓ Bonne pratique :

> Je suis bloqué sur un nouveau concept. Je demande à quelqu'un à proximité comment il l'a abordé. Nous parlons pendant 10 minutes — et soudain, ça fait clic. Je comprends.

✗ Mauvaise pratique :

> J'utilise secrètement l'IA, je copie du code qui semble correct. Pendant l'évaluation par les pairs, je ne peux rien expliquer. J'échoue. Pendant l'examen — sans IA — je suis à nouveau bloqué. J'échoue.

© 2026 Association 42. - Tous droits réservés

Chapitre 2

Préambule

Voici un extrait de dialogue issu de la série Silicon Valley :

- Je veux dire, pourquoi ne pas juste utiliser Vim au lieu d'Emacs ? (RIT)
- J'utilise Vim plutôt qu'Emacs.
- Oh mon dieu, pitié ! Ok, tu sais quoi ? Je pense que ça ne va pas marcher. Désolé. Je veux dire, quoi, on va avoir des enfants avec ça sur la conscience ? C'est pas juste pour eux, tu crois pas ?
- Des enfants ? On n'a même pas encore couché ensemble.
- Et devine quoi, ça n'arrivera jamais maintenant, parce que je ne peux pas être avec quelqu'un qui utilise des espaces au lieu des tabulations.
- Richard ! (APPUYE PLUSIEURS FOIS SUR LA BARRE ESPACE)
- Wow. Ok. Au revoir.
- Une tabulation te sauve huit espaces ! - (PORTE QUI CLAQUE) - (FRACAS)

. . .

(RICHARD GÉMIT)

- Oh mon dieu ! Richard, que s'est-il passé ?
- J'ai juste essayé de descendre les escaliers huit marches à la fois. Ça va aller.
- À bientôt, Richard.
- C'était pour faire un point.

Heureusement, vous n'êtes pas obligé d'utiliser Emacs ou la barre d'espace pour compléter les exercices suivants.

© 2026 Association 42. - Tous droits réservés

Chapitre 3

Tutoriel

- Parlons d'un des grands classiques de la programmation : la **récursivité**. Vous devriez maintenant être familier avec la chronologie d'exécution de votre code. Lorsque votre programme est exécuté, il commence avec la fonction `main`.
- Si votre fonction `main` appelle une autre fonction `F`, la fonction `main` est suspendue. Les paramètres de `F` sont calculés et mis à disposition dans le code de `F`. Lorsque `F` se termine, elle retourne une valeur, et la fonction `main` suspendue continue.
- Que se passe-t-il si `F` appelle également une autre fonction `F` ? À tout moment, une seule fonction est exécutée à la fois, et vous avez une chaîne de fonctions suspendues, commençant par `main`.



Prenez une pause ici et assurez-vous de comprendre ce comportement d'exécution avant de continuer ! Validez votre compréhension avec l'un de vos pairs.

- Maintenant, ajoutons une torsion : que se passe-t-il si votre fonction `F` appelle… la fonction `F` elle-même ? C'est tout à fait possible en C, et cela s'appelle la **récursivité**. Selon notre chronologie d'exécution, cela signifie que `F` est suspendue, ses variables existent toujours en RAM, et les variables locales de `F` - la seconde - sont créées.
- Le point clé est de comprendre que l'ordinateur agit comme si les première et seconde fonctions `F` étaient complètement différentes. Les variables de la première fonction `F` seront différentes en mémoire des variables de la seconde fonction `F`.
- Créez un répertoire de travail et écrivez une fonction récursive simple qui s'appelle elle-même depuis `main` :

main.c

```
void f()
{
    f();
}
```

Compilez et exécutez ce programme. Que se passe-t-il ? Votre programme devrait planter très rapidement.

© 2026 Association 42. - Tous droits réservés



Nous ne pouvons pas avoir une chaîne infinie de fonctions suspendues, car chacune demande de la mémoire et la mémoire est finie.

- Ajoutez un compteur pour voir combien de fois la fonction est appelée avant de planter :

main.c

```
void f()
{
    static int count = 0;
    count++;
    printf(%d\n, count);
    f();
}
```

- Exécutez à nouveau et notez le dernier nombre imprimé. C'est à peu près le nombre d'appels récursifs que votre système peut gérer.
- Maintenant, empêchez le plantage en arrêtant la récursion avant la limite. Utilisez votre compteur pour arrêter d'appeler la fonction à nouveau :

main.c

```
void f()
{
    static int count = 0;
    count++;
    if (count < 1000) // Utilisez un nombre inférieur à votre
limite de plantage
        f();
}
```

- Votre programme devrait maintenant se terminer normalement. Testez-le.
- Supprimez l'impression à l'intérieur de votre fonction. En utilisant un seul `printf` dans `main`, montrez le nombre d'appels récursifs en utilisant la valeur de retour :

© 2026 Association 42. - Tous droits réservés

main.c

```
int f()
{
    static int count = 0;
    count++;
    if (count < 1000)
        return (1 + f());
    return (1);
}
```

- La fonction doit calculer le total à travers les valeurs de retour des appels récursifs, et non simplement retourner un nombre fixe.
- Chaque appel récursif a son propre ensemble de paramètres et de variables locales. Ils n'interfèrent pas les uns avec les autres même s'ils sont dans la même fonction.

© 2026 Association 42. - Tous droits réservés

Chapitre 4

Exercice 0 : ft_iterative_factorial

	Exercice: 0	
ft_iterative_factorial		
Répertoire: *ex0/*		
Fichiers à Rendre: `ft_iterative_factorial.c`		
Autorisé: `None`		

- Écrire une fonction itérative qui retourne un entier. Cet entier est le résultat de la factorielle du nombre passé en paramètre.
- Si l'argument est invalide, la fonction devra retourner 0.
- Les dépassements de capacité (`overflow`) ne doivent pas être gérés, le comportement est alors indéfini.

Prototype :

```
int ft_iterative_factorial(int nb);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 5

Exercice 1 : ft_recursive_factorial

	Exercice: 1	
ft_recursive_factorial		
Répertoire: *ex1/*		
Fichiers à Rendre: `ft_recursive_factorial.c`		
Autorisé: `None`		

- Écrire une fonction récursive qui retourne la factorielle du nombre passé en paramètre.
- Si l'argument est invalide, la fonction devra retourner 0.
- Les dépassements de capacité ne doivent pas être gérés.

Prototype :

```
int ft_recursive_factorial(int nb);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 6

Exercice 2 : ft_iterative_power

	Exercice: 2	
ft_iterative_power		
Répertoire: *ex2/*		
Fichiers à Rendre: `ft_iterative_power.c`		
Autorisé: `None`		

- Écrire une fonction itérative qui retourne le résultat d'une puissance.
- Une puissance négative retournera 0.
- Les dépassements de capacité ne doivent pas être gérés.
- Nous avons décidé que `0^0` retournera `1`.

Prototype :

```
int ft_iterative_power(int nb, int power);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 7

Exercice 3 : ft_recursive_power

	Exercice: 3	
ft_recursive_power		
Répertoire: *ex3/*		
Fichiers à Rendre: `ft_recursive_power.c`		
Autorisé: `None`		

- Écrire une fonction récursive qui retourne le résultat d'une puissance.
- Une puissance négative retournera 0.
- Les dépassements de capacité ne doivent pas être gérés.
- `0^0` retournera `1`.

Prototype :

```
int ft_recursive_power(int nb, int power);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 8

Exercice 4 : ft_fibonacci

	Exercice: 4	
ft_fibonacci		
Répertoire: *ex*4/		
Fichiers à Rendre: `ft_fibonacci.c`		
Autorisé: `None`		

- Écrire une fonction `ft_fibonacci` qui retourne le `n`-ième élément de la suite de Fibonacci, le premier élément étant à l'indice 0. La suite est définie comme suit : 0, 1, 1, 2, 3, ...
- Les dépassements de capacité ne doivent pas être gérés.
- La fonction doit être récursive.
- Si l'indice est négatif, la fonction devra retourner `-1`.

Prototype :

```
int ft_fibonacci(int index);
```

© 2026 Association 42. - Tous droits réservés

Chapitre 9

Rendu et évaluation par les pairs

Déposez vos fichiers dans votre dépôt `Git` comme d'habitude. Seuls les fichiers présents dans votre dépôt seront pris en compte lors de la soutenance. N'hésitez pas à vérifier deux fois les noms des fichiers.



Vous devez soumettre uniquement les fichiers demandés par le sujet de ce projet.

© 2026 Association 42. - Tous droits réservés